Computational Complexity Classes

P, NP, NP-hardness, NP-completeness and beyond

Course notes

November 2026

Contents

Overview **2**

1 Preliminaries: problems, encodings and running time **2**
1.1 Decision problems and languages . . . 2
1.2 Input size and efficient algorithms . . . 3
1.3 Deterministic and nondeterministic machines . . . 3

2 The class P **3**

3 The class NP **4**
3.1 Two equivalent definitions . . . 4
3.2 Examples, step by step . . . 4
3.3 First inclusions . . . 5
3.4 The P versus NP question . . . 5

4 The class coNP **5**

5 Polynomial-time reductions **5**

6 NP-hard and NP-complete problems **6**
6.1 A recipe to prove that a problem C is NP-complete . . . 6
6.2 The first NP-complete problem: Cook–Levin . . . 7
6.3 The standard chain of reductions . . . 7

7 Worked reductions **7**
7.1 Sat $\leq_p$ 3-Sat . . . 7
7.2 3-Sat $\leq_p$ Independent-Set . . . 8
7.3 Independent-Set $\leq_p$ Vertex-Cover and Independent-Set $\leq_p$ Clique . . . 9
7.4 Other classical NP-complete problems . . . 9

8 Consequences and refinements **10**
8.1 If P $\neq$ NP: Ladner's theorem . . . 10
8.2 NP-hard but not in NP . . . 10
8.3 Weak and strong NP-completeness . . . 10
8.4 Search versus decision: self-reducibility . . . 10

9 The wider picture: space classes and the hierarchy **11**
9.1 Space-bounded classes 11
9.2 PSPACE-completeness: quantified Boolean formulas 11
9.3 Counting classes 12
9.4 Summary table 12

10 Coping with NP-hardness in practice **12**

11 Exercises **13**

12 Solutions **13**

Further reading **14**

Overview

Some problems can be solved efficiently by a computer (sorting a list, finding a shortest path), while others seem to require an exhaustive search (scheduling, packing, satisfying logical constraints). Complexity theory gives a precise language to separate these situations. In these notes we build, step by step:

1. the notion of a *decision problem* and of "efficient" computation;
2. the classes P and NP, and the P versus NP question;
3. *polynomial-time reductions*, which let us compare the difficulty of problems;
4. NP-hard and NP-complete problems, the Cook–Levin theorem, and fully worked reductions;
5. the classes around NP: coNP, PSPACE, EXP, and the known inclusions;
6. what to do in practice when a problem is NP-hard (this is where constraint programming and SAT solvers come in).

Exercises with solutions are given at the end.

1 Preliminaries: problems, encodings and running time

1.1 Decision problems and languages

Definition 1.1 (Decision problem)**.** A *decision problem* is a question whose answer is YES or NO for every input. Formally, it is a language $L \subseteq \{0,1\}^*$: the set of all (encodings of) YES-instances. Solving the problem means deciding, given a string x, whether $x \in L$.

Example 1.2.
- PATH: given a graph G and vertices s, t, is there a path from s to t?
- SAT: given a Boolean formula φ, is there an assignment of the variables making φ true?
- COMPOSITE: given an integer N, is N composite (i.e. not prime and > 1)?

Why only YES/NO questions? It simplifies the theory, and most optimisation problems have a decision version that is essentially as hard. For example, the optimisation problem "find the largest clique in G" becomes "does G contain a clique of size at least k?". If we can solve the decision version in polynomial time, we can solve the optimisation version by trying $k = 1, 2, \ldots, n$ (see Exercise 11.1).

1.2 Input size and efficient algorithms

The *size* $|x|$ of an input is the number of bits needed to write it down. This matters: an integer N has size $\approx \log_2 N$, not N. A graph with n vertices has size about n^2 with an adjacency matrix.

Definition 1.3 (Polynomial time)**.** An algorithm runs in *polynomial time* if there is a constant c such that on every input x it stops after at most $|x|^c + c$ steps (on a Turing machine or any reasonable model of computation).

We call polynomial-time algorithms "efficient". This is a convention, but a robust one:

- it does not depend on the machine model (Turing machine, RAM, etc.: they simulate each other with a polynomial overhead);
- polynomials are closed under addition, multiplication and composition, so efficient algorithms can be combined;
- in practice, n^2 or n^3 is feasible for large n, while 2^n is hopeless already at $n = 100$.

	$n = 10$	$n = 50$	$n = 100$	$n = 200$
n^2	100	2 500	10 000	40 000
n^3	1 000	125 000	10^6	$8 \cdot 10^6$
2^n	1 024	$\approx 10^{15}$	$\approx 10^{30}$	$\approx 10^{60}$

1.3 Deterministic and nondeterministic machines

A *deterministic* Turing machine (DTM) has, at each step, exactly one possible next move. A *nondeterministic* Turing machine (NTM) may have several; it *accepts* an input if *at least one* sequence of choices leads to acceptance. You can picture it as a machine that "guesses" a lucky sequence of choices. An NTM is a mathematical tool, not a realistic computer.

Definition 1.4 (Time classes)**.** Let $T : \mathbb{N} \to \mathbb{N}$.

$$\mathsf{DTIME}(T(n)) = \{L : L \text{ is decided by a deterministic TM in time } O(T(n))\},$$
$$\mathsf{NTIME}(T(n)) = \{L : L \text{ is decided by a nondeterministic TM in time } O(T(n))\}.$$

2 The class P

Definition 2.1 (P)**.**

$$\mathsf{P} = \bigcup_{k \geq 1} \mathsf{DTIME}(n^k).$$

P is the class of decision problems solvable by a deterministic algorithm in polynomial time.

Example 2.2 (Problems in P)**.**

- PATH (breadth-first search, $O(n + m)$).
- Sorting-related decisions, matching in bipartite graphs, linear programming (Khachiyan, 1979).
- 2-SAT (via strongly connected components of the implication graph).
- PRIMALITY (Agrawal–Kayal–Saxena, 2002).
- Shortest path, minimum spanning tree, maximum flow.

Closure. P is closed under complement (swap YES and NO), union, intersection, and concatenation.

3 The class NP

3.1 Two equivalent definitions

Definition 3.1 (NP, via nondeterminism)**.**

$$\mathsf{NP} = \bigcup_{k \geq 1} \mathsf{NTIME}(n^k).$$

The name means *Nondeterministic Polynomial time* (not "non-polynomial"!). The second definition is more useful in practice.

Definition 3.2 (NP, via certificates)**.** A language L is in NP if there exist a polynomial p and a deterministic polynomial-time algorithm V (the *verifier*) such that for every x:

$$x \in L \iff \exists\, w \in \{0,1\}^{\leq p(|x|)} \text{ such that } V(x,w) = \text{accept}.$$

The string w is called a *certificate* (or *witness*, or *proof*) for x.

Theorem 3.3. *The two definitions of* NP *are equivalent.*

Proof idea. ($\Leftarrow$) An NTM guesses the certificate w bit by bit ($p(|x|)$ steps), then runs the verifier $V(x,w)$. It accepts iff some guess makes V accept.
($\Rightarrow$) Take as certificate the sequence of choices made by an accepting computation path of the NTM. The verifier simulates the NTM following these choices, which takes polynomial time. □

Intuition. Problems in NP are those where a YES answer can be *checked* quickly once someone gives you a proof, even if *finding* the proof may be hard. Note the asymmetry: only YES-instances need short certificates.

3.2 Examples, step by step

Example 3.4 (SAT $\in$ NP)**.** Instance: a Boolean formula φ over variables $x_1, \ldots, x_n$.
Certificate: an assignment $a \in \{0,1\}^n$.
Verifier: evaluate φ on a. Take $\varphi = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3)$ and the certificate $a = (1,1,1)$:

1. Clause 1: $x_1 \vee \neg x_2 = 1 \vee 0 = 1$.
2. Clause 2: $\neg x_1 \vee x_3 = 0 \vee 1 = 1$.
3. Both clauses are true, so the verifier accepts.

Evaluating takes time linear in $|\varphi|$, hence SAT $\in$ NP.

Example 3.5 (CLIQUE $\in$ NP)**.** Instance: a graph G and an integer k. Question: does G contain k pairwise adjacent vertices?
Certificate: a set S of k vertices. Verifier: check that $|S| = k$ and that each of the $\binom{k}{2}$ pairs in S is an edge, in $O(k^2)$ lookups.

Example 3.6 (HAM-CYCLE $\in$ NP)**.** Certificate: an ordering $v_1, \ldots, v_n$ of the vertices. Verifier: check it is a permutation, and that $v_i v_{i+1}$ and $v_n v_1$ are edges.

Example 3.7 (SUBSET-SUM $\in$ NP)**.** Instance: integers $a_1, \ldots, a_n$ and a target T. Question: is there a subset of the a_i summing to T?
Certificate: the subset. Verifier: add its elements and compare with T. For instance, with $(3, 34, 4, 12, 5, 2)$ and $T = 9$ the certificate $\{4, 5\}$ works: $4 + 5 = 9$.

Example 3.8 (COMPOSITE $\in$ NP)**.** Certificate: a non-trivial divisor d of N. Verifier: check $1 < d < N$ and $d \mid N$.

3.3 First inclusions

Proposition 3.9. $\mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{EXP}$.

Proof. $\mathsf{P} \subseteq \mathsf{NP}$: the verifier can ignore the certificate and just solve the problem.
$\mathsf{NP} \subseteq \mathsf{EXP}$: enumerate all $2^{p(|x|)}$ certificates and run the verifier on each one; this takes time $2^{p(|x|)} \cdot \mathrm{poly}(|x|)$. □

3.4 The P versus NP question

It is not known whether $\mathsf{P} = \mathsf{NP}$. This is the most famous open problem of computer science, one of the Clay Millennium Prize problems.

- $\mathsf{P} = \mathsf{NP}$ would mean: every problem whose solutions can be checked quickly can also be solved quickly.
- Most researchers believe $\mathsf{P} \neq \mathsf{NP}$: decades of work on thousands of NP-complete problems have produced no polynomial algorithm for any of them.

4 The class coNP

Definition 4.1 (coNP)**.** $\mathsf{coNP} = \{L : \overline{L} \in \mathsf{NP}\}$, where $\overline{L}$ is the complement of L.

Equivalently, $L \in \mathsf{coNP}$ if *NO*-instances have short certificates checkable in polynomial time. Examples:

- UNSAT (is φ unsatisfiable?) $\in \mathsf{coNP}$.
- TAUTOLOGY (is φ true for all assignments?) $\in \mathsf{coNP}$: a falsifying assignment is a certificate for NO.
- PRIME $\in \mathsf{coNP}$ (the complement of COMPOSITE); in fact it is in P.

Warning. coNP is *not* the complement of NP as a set of languages. It is the set of complements of languages of NP. Since P is closed under complement, $\mathsf{P} \subseteq \mathsf{NP} \cap \mathsf{coNP}$.

Class	YES-instances have	NO-instances have
P	fast algorithm	fast algorithm
NP	short checkable certificate	(nothing required)
coNP	(nothing required)	short checkable certificate

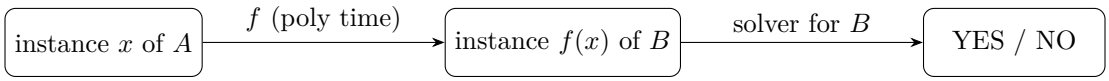
Open question: is $\mathsf{NP} = \mathsf{coNP}$? It is widely believed that it is not. If $\mathsf{NP} \neq \mathsf{coNP}$ then $\mathsf{P} \neq \mathsf{NP}$ (because P is closed under complement).

5 Polynomial-time reductions

How do we compare the difficulty of two problems without knowing how hard either one is? We transform one into the other.

Definition 5.1 (Karp reduction)**.** A language A is *polynomial-time (many-one) reducible* to B, written $A \leq_p B$, if there is a function $f : \{0,1\}^* \to \{0,1\}^*$ computable in polynomial time such that

$$\forall x : \quad x \in A \iff f(x) \in B.$$



Reading $A \leq_p B$: "A is no harder than B" (up to polynomial factors). A solver for B gives a solver for A.

Proposition 5.2 (Properties of $\leq_p$)**.** *(a)* Transitivity: *$A \leq_p B$ and $B \leq_p C$ imply $A \leq_p C$.*
(b) If $A \leq_p B$ and $B \in \mathsf{P}$, then $A \in \mathsf{P}$.
(c) If $A \leq_p B$ and $B \in \mathsf{NP}$, then $A \in \mathsf{NP}$.
(d) If $A \leq_p B$ and $A \notin \mathsf{P}$, then $B \notin \mathsf{P}$.

Proof. (a) If f reduces A to B and g reduces B to C, then $g \circ f$ reduces A to C. It is computable in polynomial time: $|f(x)| \leq \text{poly}(|x|)$, so computing $g(f(x))$ costs $\text{poly}(\text{poly}(|x|))$, which is again polynomial.
(b) To decide $x \in A$: compute $f(x)$ in polynomial time, then run the polynomial-time algorithm for B on it. The total is a composition of polynomials.
(c) Same idea with the certificate for $f(x)$.
(d) Contrapositive of (b). □

6 NP-hard and NP-complete problems

Definition 6.1 (NP-hard, NP-complete)**.** Let B be a language.

- B is NP-*hard* if $A \leq_p B$ for *every* $A \in \mathsf{NP}$.
- B is NP-*complete* if B is NP-hard *and* $B \in \mathsf{NP}$.

So NP-complete problems are the *hardest problems inside* NP: every problem of NP reduces to them. NP-hard problems are at least as hard as everything in NP, but they may live outside NP (and need not even be decision problems; they can be optimisation problems).

Theorem 6.2. *If some NP-complete problem is in P, then P = NP.*

Proof. Let B be NP-complete with $B \in \mathsf{P}$. For any $A \in \mathsf{NP}$ we have $A \leq_p B$, so $A \in \mathsf{P}$ by Proposition 5.2(b). Hence $\mathsf{NP} \subseteq \mathsf{P}$, and $\mathsf{P} \subseteq \mathsf{NP}$ always holds. □

Lemma 6.3 (Propagating hardness)**.** *If B is NP-hard and $B \leq_p C$, then C is NP-hard. If moreover $C \in \mathsf{NP}$, then C is NP-complete.*

Proof. For any $A \in \mathsf{NP}$, $A \leq_p B$ and $B \leq_p C$, so $A \leq_p C$ by transitivity. □

6.1 A recipe to prove that a problem C is NP-complete

Step 1. Show $C \in \mathsf{NP}$. Describe a certificate and a polynomial-time verifier.

Step 2. Choose a known NP-complete problem B. Choose one that looks similar to C.

Step 3. Build a polynomial-time map f sending every instance x of B to an instance $f(x)$ of C.

Step 4. Prove correctness in both directions: $x \in B \Rightarrow f(x) \in C$ and $f(x) \in C \Rightarrow x \in B$.

Step 5. Prove that f is polynomial-time (size of $f(x)$ and time to compute it).

A frequent mistake is to reduce in the wrong direction: you must reduce the *known hard* problem *to* your new problem, not the other way round.

6.2 The first NP-complete problem: Cook–Levin

Lemma 6.3 needs a first NP-hard problem to start the chain. This is provided by:

Theorem 6.4 (Cook 1971, Levin 1973)**.** *Sat is* NP*-complete.*

Proof sketch. Sat $\in$ NP was shown above. For hardness, let $A \in \mathsf{NP}$, decided by an NTM M in time $p(n)$ on inputs of length n. Fix an input x with $|x| = n$. We build in polynomial time a formula φ_x that is satisfiable iff M accepts x.

Step 1: the tableau. A computation of M on x has at most $p(n)$ steps and uses at most $p(n)$ tape cells. Write it as a table with $p(n)$ rows (time) and $p(n)$ columns (tape cells). Each cell contains one symbol from a finite alphabet Γ' (a tape symbol, possibly combined with the machine state if the head is there).

Step 2: variables. For every row i, column j and symbol $s \in \Gamma'$ introduce a Boolean variable $y_{i,j,s}$, meaning "cell (i, j) contains s". There are $O(p(n)^2)$ variables.

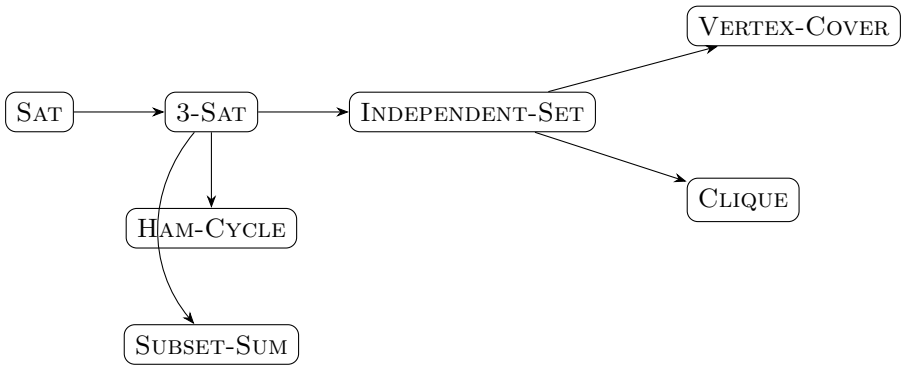
Step 3: constraints. The formula φ_x is the conjunction of

(i) *one symbol per cell:* for each (i, j), exactly one $y_{i,j,s}$ is true;
(ii) *initial row:* row 1 encodes the start state, the head on the first cell and the input x;
(iii) *legal transitions:* every 2×3 window of the table is consistent with the transition relation of M (a computation step only changes cells next to the head, so it can be checked locally);
(iv) *acceptance:* some cell of the table contains the accepting state.

Step 4: size and correctness. Each constraint involves a constant number of variables and there are $O(p(n)^2)$ of them, so $|\varphi_x| = O(p(n)^2)$ and it is computed in polynomial time. A satisfying assignment is exactly an accepting computation (the nondeterministic choices are read from the table), and conversely. Hence $x \in A \iff \varphi_x \in$ Sat. $\square$

6.3 The standard chain of reductions

From Sat, thousands of problems were shown NP-complete (Karp's 21 problems, 1972, started the list).



7 Worked reductions

We now carry out three reductions in detail, following the recipe of the previous section.

7.1 Sat $\leq_p$ 3-Sat

Definition 7.1 (3-Sat)**.** A formula is in *3-CNF* if it is a conjunction of clauses, each clause being a disjunction of *exactly three* literals. 3-Sat asks whether a 3-CNF formula is satisfiable.

We assume SAT is given in CNF (any formula can be converted to an equisatisfiable CNF in polynomial time by the Tseitin transformation). We replace each clause C by a set of 3-literal clauses depending on its length k:

Step 1. If $k = 1$, $C = (\ell)$: introduce fresh variables u, v and use

$$(\ell \vee u \vee v) \wedge (\ell \vee \neg u \vee v) \wedge (\ell \vee u \vee \neg v) \wedge (\ell \vee \neg u \vee \neg v).$$

Whatever the values of u, v, exactly the assignments with $\ell = 1$ satisfy all four clauses.

Step 2. If $k = 2$, $C = (\ell_1 \vee \ell_2)$: use a fresh u and $(\ell_1 \vee \ell_2 \vee u) \wedge (\ell_1 \vee \ell_2 \vee \neg u)$.

Step 3. If $k = 3$: keep C.

Step 4. If $k \geq 4$, $C = (\ell_1 \vee \cdots \vee \ell_k)$: introduce fresh variables $z_1, \ldots, z_{k-3}$ and use the chain

$$(\ell_1 \vee \ell_2 \vee z_1) \wedge (\neg z_1 \vee \ell_3 \vee z_2) \wedge (\neg z_2 \vee \ell_4 \vee z_3) \wedge \cdots \wedge (\neg z_{k-3} \vee \ell_{k-1} \vee \ell_k).$$

Example 7.2. Take $C = (a \vee b \vee c \vee d \vee e)$, so $k = 5$. We get

$$(a \vee b \vee z_1) \wedge (\neg z_1 \vee c \vee z_2) \wedge (\neg z_2 \vee d \vee e).$$

If C is true, say $c = 1$ and all others 0: set $z_1 = 1$ (satisfies clause 1) and $z_2 = 0$ (satisfies clause 3 via $\neg z_2$); clause 2 holds because $c = 1$.
If C is false, then $a = b = c = d = e = 0$. Clause 1 forces $z_1 = 1$, then clause 2 forces $z_2 = 1$, then clause 3 is $0 \vee 0 \vee 0$, which is false. So the chain is unsatisfiable exactly when C is.

Correctness. For a general $k \geq 4$ the argument is the same: if some $\ell_i = 1$, set $z_1, \ldots, z_{i-2} = 1$ and $z_{i-1}, \ldots, z_{k-3} = 0$. If all $\ell_i = 0$, the chain forces $z_1 = 1$, then $z_2 = 1$, …, then $z_{k-3} = 1$, falsifying the last clause. The new formula has size $O(|\varphi|)$ and is built in polynomial time. Since 3-SAT $\in$ NP (it is a special case of SAT), 3-SAT is NP-complete.

7.2 3-Sat $\leq_p$ Independent-Set

Definition 7.3 (INDEPENDENT-SET)**.** Given a graph G and an integer k, does G contain an *independent set* of size k, i.e. k vertices, no two of which are adjacent?

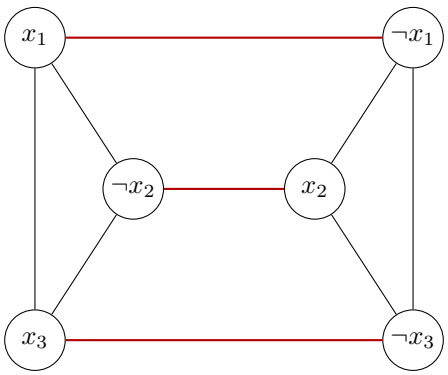
Construction. Given a 3-CNF formula φ with m clauses:

Step 1. For each clause, create a *triangle* of 3 vertices, one per literal of the clause.

Step 2. Add a *conflict edge* between any two vertices (in different triangles) labelled by complementary literals x and $\neg x$.

Step 3. Set $k = m$.

Example 7.4. Let $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$. Here $m = 2$, so $k = 2$.



The black edges are the triangles, the red edges are the conflicts. Choose x_1 in the first triangle and x_2 in the second: they are not adjacent, so $\{x_1, x_2\}$ is an independent set of size 2. It corresponds to the assignment $x_1 = 1$, $x_2 = 1$ (and x_3 arbitrary), which indeed satisfies both clauses (clause 1 by x_1, clause 2 by x_2).

Correctness.

- ($\Rightarrow$) Suppose φ is satisfiable. In every clause pick one true literal, and take the corresponding vertex. We get m vertices, one per triangle, so no triangle edge joins two of them. No conflict edge joins two of them either, since a conflict edge links x and $\neg x$ and they cannot both be true. So this is an independent set of size $m = k$.
- ($\Leftarrow$) Suppose there is an independent set S of size m. It has at most one vertex per triangle (triangle edges), hence exactly one in each of the m triangles. Set each chosen literal to true; this is consistent because no two chosen literals are complementary (conflict edges). Set the remaining variables arbitrarily. Every clause contains a true literal, so φ is satisfied.

The graph has $3m$ vertices and at most $3m + 3\binom{m}{2} \cdot 3$ edges, so the construction is polynomial. With Independent-Set $\in$ NP, we conclude that Independent-Set is NP-complete.

7.3 Independent-Set $\leq_p$ Vertex-Cover and Independent-Set $\leq_p$ Clique

Definition 7.5. A *vertex cover* of $G = (V, E)$ is a set $C \subseteq V$ such that every edge has at least one endpoint in C. A *clique* is a set of pairwise adjacent vertices.

Lemma 7.6. *For every graph $G = (V, E)$ and $S \subseteq V$: S is an independent set $\iff$ $V \setminus S$ is a vertex cover.*

Proof. S is independent $\iff$ no edge has both endpoints in S $\iff$ every edge has at least one endpoint outside S $\iff$ $V \setminus S$ is a vertex cover. □

Reduction Independent-Set $\leq_p$ Vertex-Cover. Map (G, k) to $(G, n - k)$ where $n = |V|$. By the lemma, G has an independent set of size k iff it has a vertex cover of size $n - k$. This is computed in linear time. In the previous example $n = 6$ and $k = 2$, so the corresponding vertex cover has size 4, namely all vertices except x_1 and x_2.

Reduction Independent-Set $\leq_p$ Clique. Map (G, k) to $(\overline{G}, k)$, where $\overline{G}$ has the same vertices and exactly the non-edges of G. A set is independent in G iff it is a clique in $\overline{G}$.

Since Vertex-Cover and Clique are in NP (verify a certificate set directly), both are NP-complete.

7.4 Other classical NP-complete problems

Without proof:

- Ham-Cycle and the Travelling Salesman decision problem (TSP: is there a tour of length $\leq L$?).
- Subset-Sum, Partition, Knapsack (decision version), Bin-Packing.
- 3-Colouring (but 2-Colouring is in P!).
- Set-Cover, Exact-Cover, Dominating-Set.
- 0-1 Integer Programming, generalised Sudoku on $n^2 \times n^2$ grids, Latin square completion, many scheduling and timetabling problems.

Thin line between easy and hard.

In P	NP-complete
2-SAT	3-SAT
2-COLOURING	3-COLOURING
Perfect matching in a graph	3-dimensional matching
Shortest path	Longest simple path
Eulerian cycle (each edge once)	Hamiltonian cycle (each vertex once)
Linear programming	Integer linear programming

8 Consequences and refinements

8.1 If P ≠ NP: Ladner's theorem

Theorem 8.1 (Ladner 1975)**.** *If* P ≠ NP*, then there exist languages in* NP *that are neither in* P *nor* NP*-complete (called* NP-intermediate*).*

Natural candidates for NP-intermediate problems are GRAPH ISOMORPHISM (Babai showed it is solvable in quasi-polynomial time, $2^{(\log n)^{O(1)}}$) and INTEGER FACTORISATION (decision version).

8.2 NP-hard but not in NP

NP-hard problems can be much harder than NP-complete ones.

- The *optimisation* problems "find a minimum vertex cover", "find the shortest TSP tour" are NP-hard but not decision problems, hence not in NP by definition.
- The HALTING PROBLEM is NP-hard (every NP problem reduces to it) and undecidable.
- TQBF (see below) is PSPACE-complete, hence NP-hard, and not known to be in NP.

	NP-hard	in NP	NP-complete
SAT	yes	yes	yes
Shortest path	no*	yes	no*
Min. TSP tour (optimisation)	yes	not a decision problem	no
Halting problem	yes	no (undecidable)	no

* unless P = NP.

8.3 Weak and strong NP-completeness

SUBSET-SUM can be solved by dynamic programming in time $O(n \cdot T)$, which is polynomial in the *value* of T but exponential in its *bit length* $\log T$. Such algorithms are called *pseudo-polynomial*. A problem that stays NP-hard even when all numbers are written in unary (e.g. 3-PARTITION, BIN-PACKING in general, HAM-CYCLE) is called *strongly* NP-complete, and cannot have a pseudo-polynomial algorithm unless P = NP.

8.4 Search versus decision: self-reducibility

SAT is *self-reducible*: if we have a polynomial-time algorithm that decides satisfiability, we can *find* a satisfying assignment.

Step 1. Ask: is φ satisfiable? If NO, stop.

Step 2. Fix $x_1 = 1$ (substitute and simplify) and ask again. If YES keep $x_1 = 1$, otherwise set $x_1 = 0$.

Step 3. Repeat for $x_2, \ldots, x_n$.

This uses $n+1$ calls. Hence for NP-complete problems, the search version is no harder than the decision version.

9 The wider picture: space classes and the hierarchy

9.1 Space-bounded classes

Definition 9.1. L and NL are the problems solved in $O(\log n)$ work space by a deterministic, respectively nondeterministic, machine. PSPACE (resp. NPSPACE) is the class of problems solved in polynomial space by a deterministic (resp. nondeterministic) machine. $\mathsf{EXP} = \bigcup_k \mathsf{DTIME}(2^{n^k})$ and $\mathsf{NEXP} = \bigcup_k \mathsf{NTIME}(2^{n^k})$.

Theorem 9.2 (Savitch 1970)**.** PSPACE = NPSPACE.

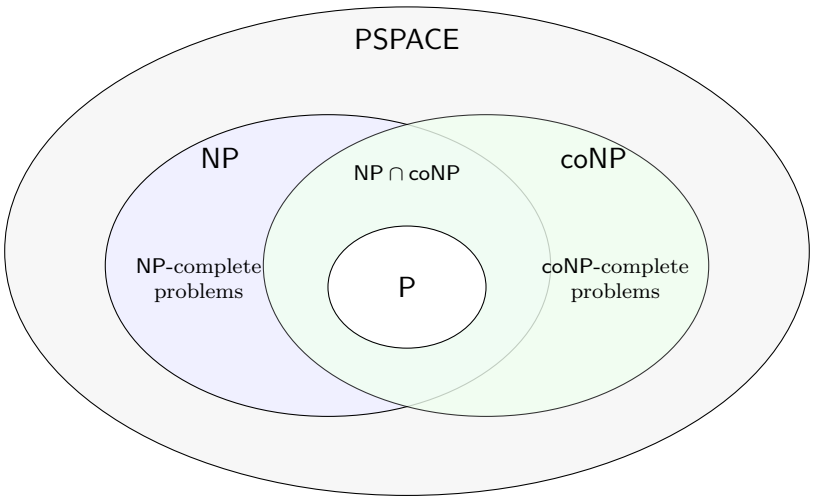
(Nondeterminism does not help for polynomial space, in contrast with the conjectured situation for time.)

Theorem 9.3 (Known inclusions)**.**

$$\mathsf{L} \subseteq \mathsf{NL} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP} \subseteq \mathsf{NEXP},$$

and coNP ⊆ PSPACE *as well.*

What is known to be strict? By the hierarchy theorems, NL ⊊ PSPACE, P ⊊ EXP and NP ⊊ NEXP. So at least one of the inclusions between P and EXP is strict, but we do not know which. In particular none of $\mathsf{L} \stackrel{?}{\neq} \mathsf{P}$, $\mathsf{P} \stackrel{?}{\neq} \mathsf{NP}$, $\mathsf{NP} \stackrel{?}{\neq} \mathsf{PSPACE}$, $\mathsf{PSPACE} \stackrel{?}{\neq} \mathsf{EXP}$ has been resolved.



The usual picture of the classes, ***assuming*** *the widely believed conjectures* P ≠ NP*,* NP ≠ coNP *and* NP ∩ coNP ≠ P*. If* P = NP*, the pictured inner regions collapse.*

9.2 PSPACE-completeness: quantified Boolean formulas

A *quantified Boolean formula* (QBF) is a formula such as

$$\exists x_1 \, \forall x_2 \, \exists x_3 \; \psi(x_1, x_2, x_3),$$

where ψ is quantifier-free. TQBF asks whether a fully quantified formula is true. Compare with SAT, which is the special case where all quantifiers are $\exists$.

Theorem 9.4. *TQBF is* PSPACE*-complete.*

Example 9.5. $\exists x\, \forall y\ (x \vee y) \wedge (\neg y \vee x)$. Try $x = 1$: for $y = 0$ the formula is $(1) \wedge (1) = 1$; for $y = 1$ it is $(1) \wedge (1) = 1$. So the sentence is true. The alternation of $\exists$ and $\forall$ models two-player games ("there is a move such that for every reply of the opponent, ..."), which is why many board games (generalised Geography, Hex, ...) are PSPACE-complete.

9.3 Counting classes

#P ("sharp-P") is the class of *counting* problems: instead of asking whether a satisfying assignment exists, count how many. #Sat is #P-complete. Some counting problems are hard even though the decision problem is easy: counting perfect matchings (permanent of a 0/1 matrix) is #P-complete although finding one is in P. Toda's theorem says the whole polynomial hierarchy reduces to #P.

9.4 Summary table

Class	Informal meaning	Typical complete problem
P	solvable quickly	(e.g. circuit value, under log-space reductions)
NP	YES verifiable quickly	Sat, 3-Sat, Clique, Ham-Cycle
coNP	NO verifiable quickly	Tautology, Unsat
PSPACE	polynomial memory	TQBF, generalised games
EXP	exponential time	generalised chess/checkers
#P	counting solutions	#Sat, permanent

10 Coping with NP-hardness in practice

Being NP-hard is a statement about the *worst case*. It does not mean that instances you care about are unsolvable. The main strategies:

1. **Exploit special structure.** Restrict to a tractable subclass (e.g. 2-Sat, planar graphs, trees, bounded treewidth, Horn clauses).
2. **Parameterised algorithms (FPT).** If a parameter k is small, run in time $f(k) \cdot n^{O(1)}$. Example: Vertex-Cover with parameter k is solvable in $O(2^k \cdot n)$ by branching on an uncovered edge uv (either u or v must be in the cover).
3. **Approximation algorithms.** Return a solution within a guaranteed factor of the optimum. Example: taking both endpoints of a maximal matching gives a vertex cover at most $2\times$ optimal. (Some problems cannot be approximated at all unless P = NP, e.g. general TSP.)
4. **Heuristics and local search.** No guarantee, often good in practice.
5. **Exact solvers with strong pruning.** Modern Sat solvers (CDCL), SMT solvers, integer programming solvers, and *constraint programming* solvers routinely solve industrial instances with millions of variables. They combine search, *propagation* (deducing consequences of constraints to prune the search tree) and learning.

Link with constraint programming. A constraint satisfaction problem (CSP) asks for an assignment of values from finite domains to variables satisfying a set of constraints. It generalises Sat (domains $\{0, 1\}$, clauses as constraints), so deciding whether a CSP has a solution is NP-complete in general. This is exactly why CP solvers rely on propagation and search heuristics rather than on a polynomial algorithm, and why identifying tractable fragments of constraint languages (the Dichotomy theorem of Bulatov and Zhuk: every finite-domain constraint language gives a CSP that is either in P or NP-complete) is an active research area.

11 Exercises

Exercise 11.1. Suppose you have a polynomial-time algorithm for the decision problem Clique. Show how to compute the *size* of a maximum clique in polynomial time, and then how to *find* a maximum clique.

Exercise 11.2. Show that $\mathsf{P} \subseteq \mathsf{NP} \cap \mathsf{coNP}$.

Exercise 11.3. Prove that if $A \leq_p B$ and $B \leq_p C$ then $A \leq_p C$, being careful about the size of the intermediate instance.

Exercise 11.4. True or false (justify): (a) every NP-hard problem is in NP; (b) if $\mathsf{P} = \mathsf{NP}$, every problem in NP (except $\emptyset$ and $\{0,1\}^*$) is NP-complete; (c) if $\mathsf{NP} \neq \mathsf{coNP}$ then $\mathsf{P} \neq \mathsf{NP}$.

Exercise 11.5. Convert $(a \vee b \vee c \vee d)$ into an equisatisfiable 3-CNF formula, and then apply the 3-Sat $\leq_p$ Independent-Set construction to $(a \vee \neg b \vee c) \wedge (\neg a \vee b \vee \neg c) \wedge (a \vee b \vee c)$. Give the graph, the value of k, and an independent set of size k.

Exercise 11.6. Show that Set-Cover is NP-complete by reduction from Vertex-Cover. (*Hint:* the universe is the set of edges; one set per vertex.)

Exercise 11.7. Show that Partition (can a multiset of integers be split into two parts with equal sums?) is in NP, and reduce it to Subset-Sum.

Exercise 11.8. Show that the Vertex-Cover problem with parameter k can be solved in time $O(2^k \cdot (n+m))$.

12 Solutions

Exercise 11.1. **Size:** for $k = n, n-1, \ldots, 1$, ask the algorithm whether a clique of size k exists; the first YES gives the maximum size k^* (n calls). **Finding it:** go through the vertices v one by one: delete v from the graph and ask whether a clique of size k^* still exists. If YES, keep v deleted; if NO, put v back and mark it as part of the clique. At the end the marked vertices form a maximum clique. This takes $O(n)$ calls.

Exercise 2. $\mathsf{P} \subseteq \mathsf{NP}$ was shown. For coNP: if $L \in \mathsf{P}$ then $\overline{L} \in \mathsf{P} \subseteq \mathsf{NP}$, so $L \in \mathsf{coNP}$.

Exercise 3. If f runs in time $|x|^a$ then $|f(x)| \leq |x|^a$ (we cannot write more symbols than steps). If g runs in time $|y|^b$, then $g(f(x))$ is computed in time $|x|^a + (|x|^a)^b = O(|x|^{ab})$, polynomial. And $x \in A \iff f(x) \in B \iff g(f(x)) \in C$.

Exercise 4. (a) False: the halting problem, or the optimisation version of TSP. (b) True: if $\mathsf{P} = \mathsf{NP}$, for any such B and any $A \in \mathsf{NP} = \mathsf{P}$ pick $y_0 \in B$, $y_1 \notin B$ and define $f(x) = y_0$ if $x \in A$, y_1 otherwise; f is polynomial-time computable since $A \in \mathsf{P}$. (c) True: if $\mathsf{P} = \mathsf{NP}$ then $\mathsf{NP} = \mathsf{P} = \mathsf{coNP}$ because P is closed under complement; contrapositive.

Exercise 5. $(a \vee b \vee c \vee d) \to (a \vee b \vee z) \wedge (\neg z \vee c \vee d)$. For the second part, call the clauses $C_1 = (a \vee \neg b \vee c)$, $C_2 = (\neg a \vee b \vee \neg c)$, $C_3 = (a \vee b \vee c)$. The graph has 9 vertices (three triangles) and $k = 3$. Conflict edges: $a_{C_1} - \neg a_{C_2}$, $\neg a_{C_2} - a_{C_3}$, $\neg b_{C_1} - b_{C_2}$, $\neg b_{C_1} - b_{C_3}$, $c_{C_1} - \neg c_{C_2}$, $\neg c_{C_2} - c_{C_3}$. The set $\{a_{C_1}, b_{C_2}, a_{C_3}\}$ is independent of size 3; it gives the assignment $a = 1$, $b = 1$, which satisfies all three clauses.

Exercise 6. Set-Cover $\in \mathsf{NP}$ (check the chosen sets). Given $(G = (V, E), k)$, let the universe be E and for each $v \in V$ let S_v be the set of edges incident to v. A vertex cover of size k is exactly a choice of k sets S_v covering E. The map is clearly polynomial.

Exercise 7. Certificate: one of the two parts; the verifier sums both. Reduction PARTITION $\leq_p$ SUBSET-SUM: given $(a_1, \ldots, a_n)$ with total S, output the same numbers with target $T = S/2$ (output a trivial NO instance if S is odd). A subset sums to $S/2$ iff its complement does too, so it splits the multiset in two equal parts.

Exercise 8. If G has no edges, answer YES. Otherwise pick any edge uv. Any vertex cover contains u or v. Recurse on $(G - u, k - 1)$ and $(G - v, k - 1)$; stop with NO when $k = 0$ and edges remain. The recursion tree has depth at most k and branching factor 2, hence at most 2^k leaves, and each node costs $O(n + m)$.

Further reading

- S. Arora and B. Barak, *Computational Complexity: A Modern Approach*, Cambridge University Press, 2009.
- M. Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage, 2012.
- M. Garey and D. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, 1979.
- C. Papadimitriou, *Computational Complexity*, Addison-Wesley, 1994.